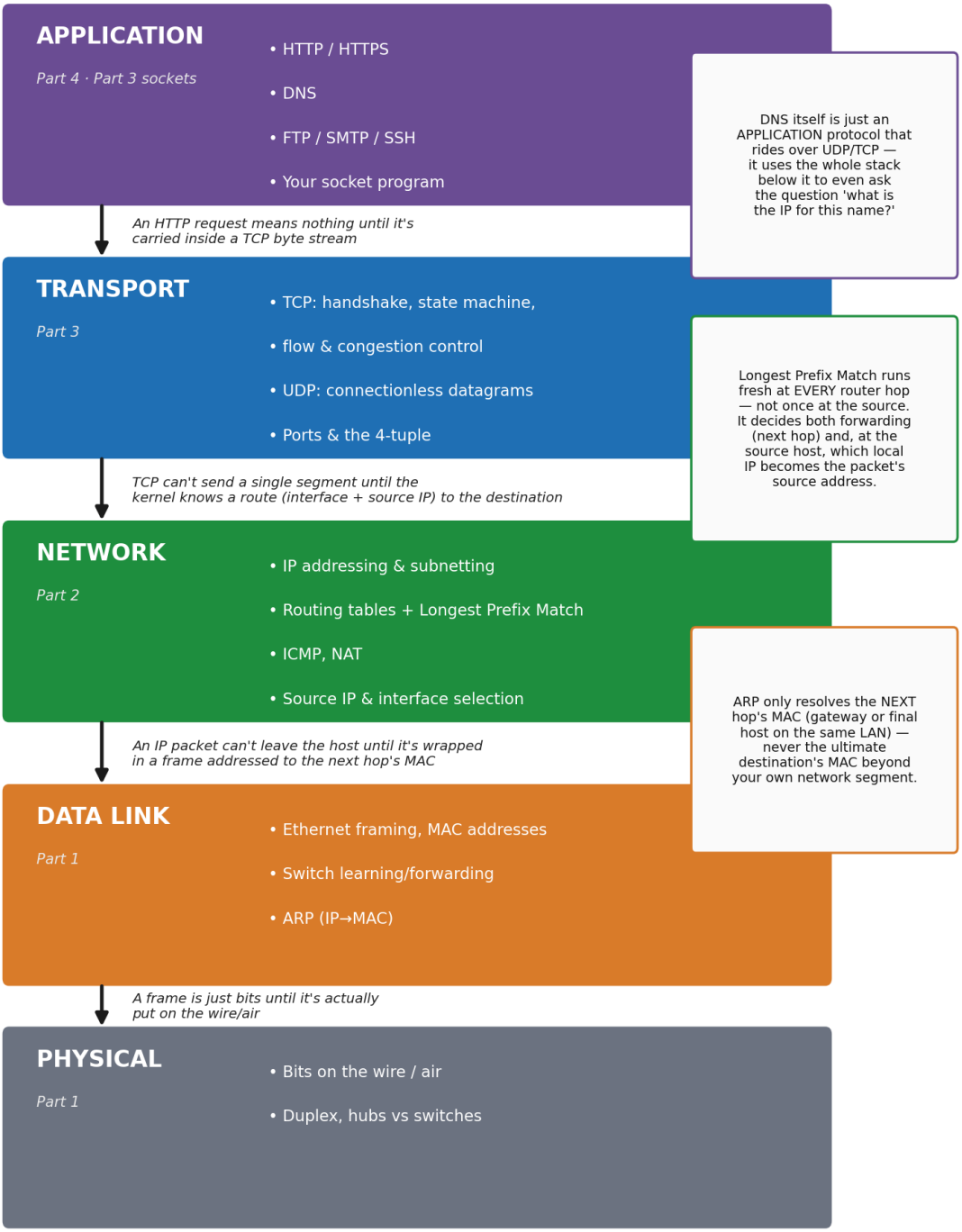
Computer Networks — The Big Picture

A visual map of how every topic from Parts 1–4 connects, plus worked examples — including the full journey of a single browser request from keystroke to rendered page.

How to use this PDF: it's meant to be read alongside Parts 1–4, not instead of them. Every box below references the section that explains it in full (e.g. “Part 2 §3”). Start with the map on the next page to see the whole shape of the subject, then follow the journey diagrams to see that shape in action on one real request.

How Every Topic Connects — The Layer Dependency Map

Each layer only works because the layer below it delivers something specific.
Read bottom to top: physical bits enable frames, frames enable packets, packets enable segments, segments enable your app.



Worked Example — "I typed a URL, what actually happens?"

This is the request your professor's routing-table question was really pointing at: every mechanism from Parts 1–4 fires, in this order, for something as simple as loading a page. Two diagrams below split the journey into (A) setting up a secure connection, and (B) the request/response and cleanup.

The Journey of One Browser Request — Part A

From pressing Enter to having a secure TCP connection ready for the HTTP request

1. **You type example.com and press Enter**
   Browser checks its own cache, then asks the OS to resolve the name.
2. **OS checks local DNS cache**
   If a cached A record hasn't expired (its TTL), skip straight to step 5.
3. **Recursive resolver walks the DNS hierarchy**
   Root → .com TLD server → example.com's authoritative server, over UDP/53 (Part 4 §1).
4. **Authoritative server returns the A record**
   e.g. 93.184.216.34, cached locally for its TTL.
5. **Kernel runs Longest Prefix Match**
   Looks up 93.184.216.34 in the routing table; the most specific matching route wins, giving an outgoing interface + next-hop gateway (Part 2 §3).
6. **Kernel picks the source IP**
   Taken from the address(es) on that chosen interface — this is the literal answer to 'how is my IP decided' (Part 2 §4).
7. **ARP resolves the gateway's MAC**
   Checked in the ARP cache first; if missing, a broadcast ARP request/reply fills it in (Part 1 §4.6).
8. **TCP three-way handshake**
   SYN → SYN-ACK → ACK to the server's IP:443, each side picking a random Initial Sequence Number (Part 3 §3.3).
9. **Every handshake segment is IP-wrapped and hops router to router**
   Each router independently re-runs its OWN longest-prefix-match; TTL drops by 1 each hop (Part 2 §3.4).
10. **At each hop, link-layer framing + local ARP**
    Resolves only the NEXT hop's MAC, never the final server's, until the very last LAN segment (Part 1 §4.6).
11. **TLS handshake**
    Runs on top of the now-ESTABLISHED TCP connection, negotiating encryption before any HTTP bytes go out (Part 4 §2.5).

The Journey of One Browser Request — Part B

From sending the HTTP request to a rendered page and closed connection

12

Browser sends the HTTP GET request

Sent inside the encrypted TCP stream; includes the mandatory Host header so the right virtual site is served (Part 4 §2.2).

13

Server streams the response back

TCP's flow control (receiver's advertised window) and congestion control (cwnd, slow start → congestion avoidance) jointly govern how fast bytes can arrive (Part 3 §3.5-3.6).

14

Browser needs more objects (CSS, JS, images)

Reuses the same connection via HTTP/1.1 keep-alive or HTTP/2 multiplexed streams, or opens new connections — each new destination repeats steps 5–11 (Part 4 §2.4).

15

Page renders

Browser has a complete, in-order byte stream to parse — guaranteed by TCP's sequence/ack numbers, not by anything the application did itself.

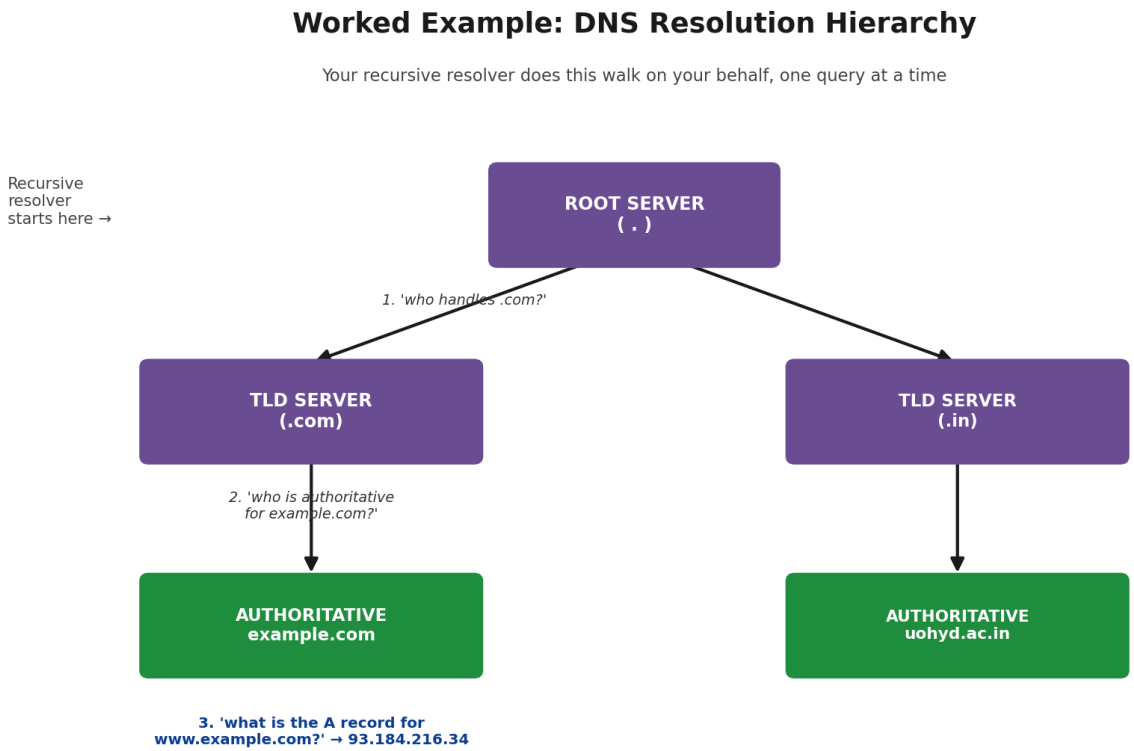
16

Connection teardown

Four-way FIN/ACK close, or left idle in keep-alive; the active closer enters TIME_WAIT for ~2×MSL before fully releasing the connection (Part 3 §3.4).

Worked Example — DNS Resolution

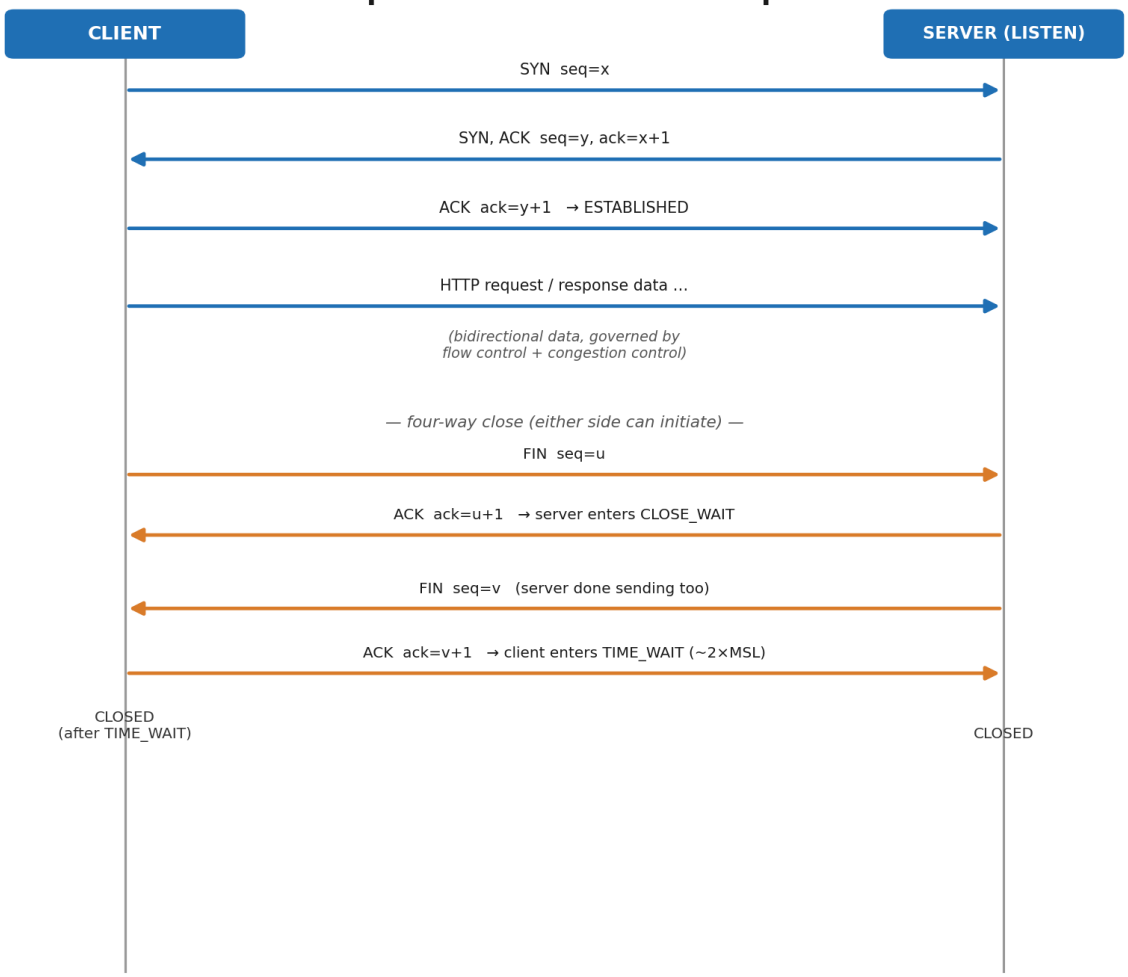
Step 3 of the journey above compresses into one box what's actually a multi-hop conversation. This is that conversation expanded.



Worked Example — TCP Setup & Teardown

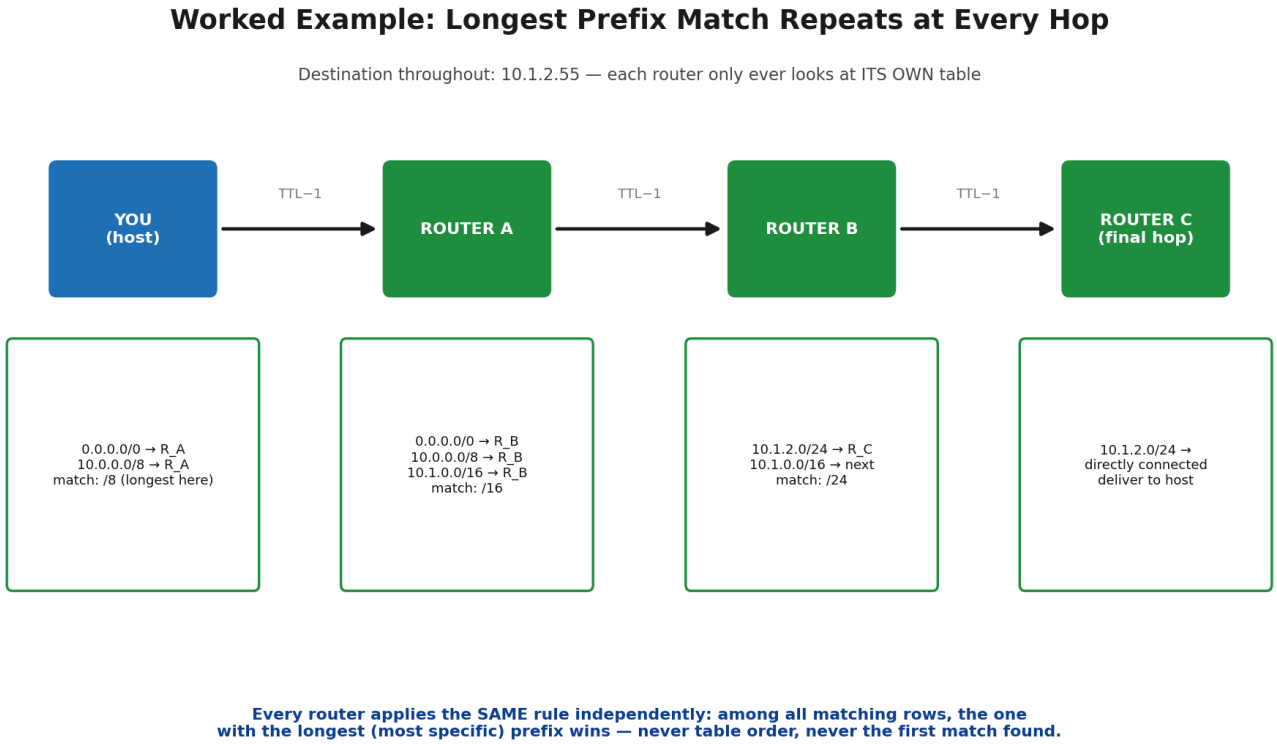
Step 8 of the journey (and the four-way close at the end) as an explicit sequence diagram, with the exact sequence/ack numbers and the state each side lands in.

Worked Example: TCP Connection Setup & Teardown



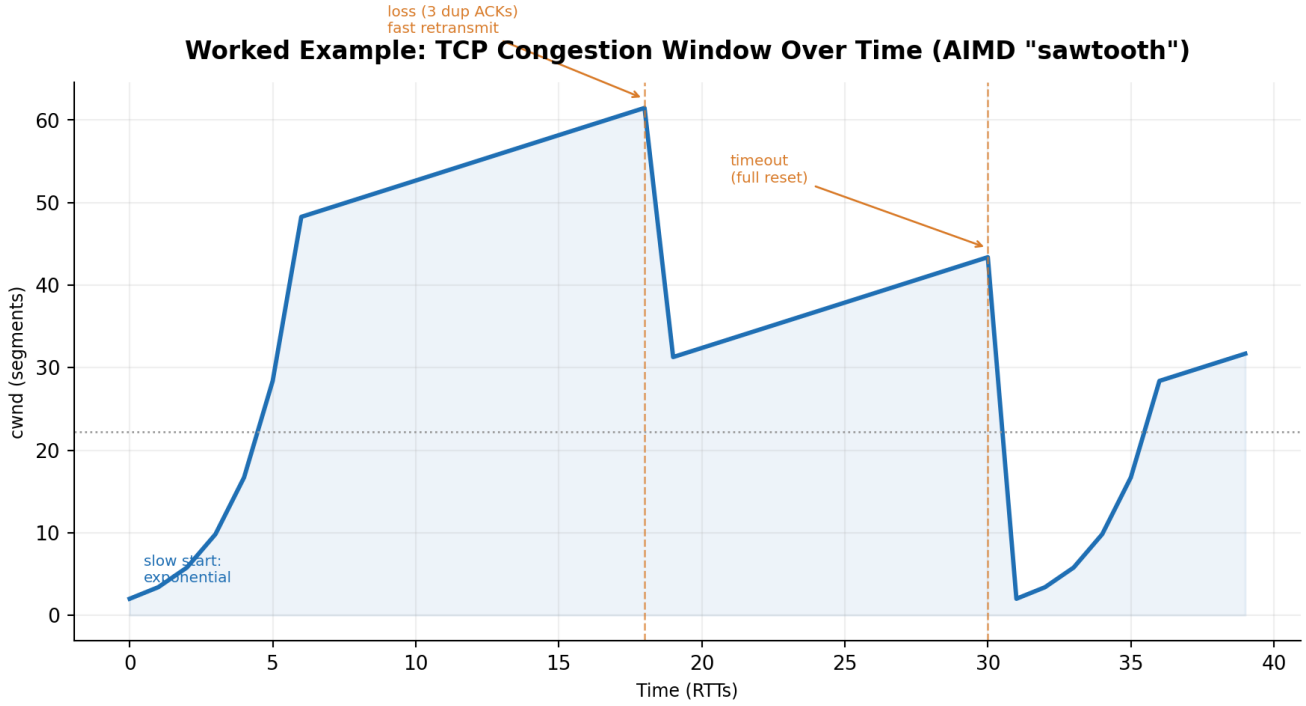
Worked Example — Longest Prefix Match, Hop by Hop

Step 9 of the journey said each router re-runs its own lookup. This shows the same destination, 10.1.2.55, producing a DIFFERENT winning route at each router — because each router only ever has its own table, never a global view.



Worked Example — TCP Congestion Window Over Time

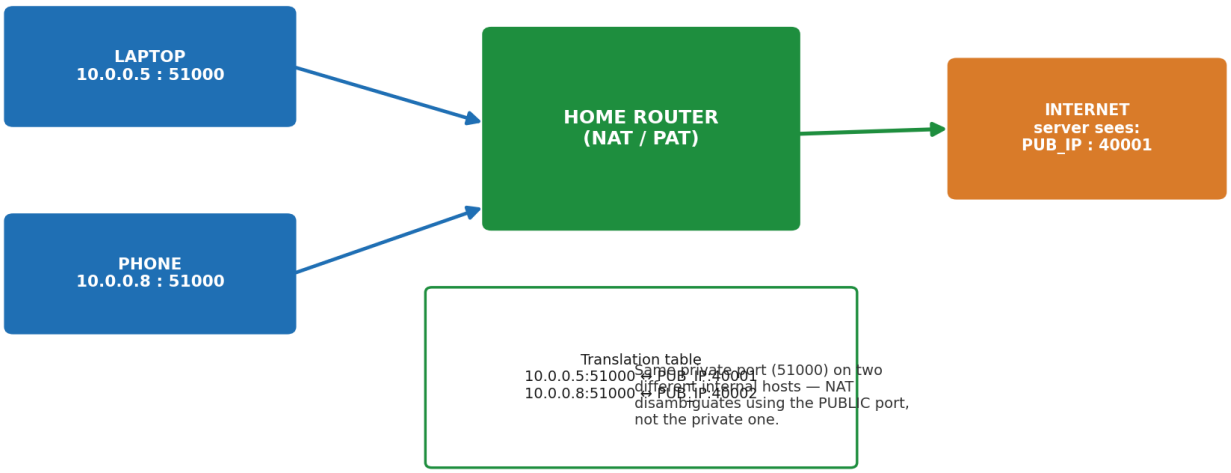
Step 13 of the journey mentions congestion control governing pace. This is what that actually looks like plotted over time: exponential growth (slow start), linear growth (congestion avoidance), and the two different reactions to loss from Part 3 §3.6.


Worked Example: TCP Congestion Window Over Time (AIMD "sawtooth")
loss (3 dup ACKs)
fast retransmit
timeout
(full reset)
slow start:
exponential
cwnd (segments)
Time (RTTs)
0
10
20
30
40
50
60
0
5
10
15
20
25
30
35
40

Worked Example — NAT in Action

Not part of the browser journey directly, but the mechanism your home network is running on every one of those connections simultaneously — rewriting the source half of the 4-tuple from Part 3 §1, using the translation table concept from Part 2 §7.

Worked Example: NAT Rewrites the Source Half of the 4-Tuple



How to talk about this in class: when she asks about any single mechanism, place it on the master map first (which layer, which Part), then narrate one step earlier and one step later using the journey diagram. That's the difference between reciting a definition and showing you understand the whole picture.